Radar Carrière

AI internship radar on Telegram · n8n Scoping Deliverable · Target: Level 2 with Level 3 features

Project overview. Radar Carrière searches for job and internship offers every night, has an AI score each one against the student's profile, and sends the best ones on Telegram every morning. The student answers with one tap (Yes / No / Maybe); on "Yes", the AI writes a cover letter adapted to the offer. The student keeps the final decision: nothing is sent to an employer automatically.

1. The need — in one sentence

Students looking for an internship spend hours every week browsing job sites one by one and tracking offers in a spreadsheet, often missing the relevant ones or their deadlines; Radar Carrière collects offers automatically, has an AI score each one against the student's profile, and sends the best ones on Telegram every morning so the student can decide in one tap.

2. The trigger

Main triggers

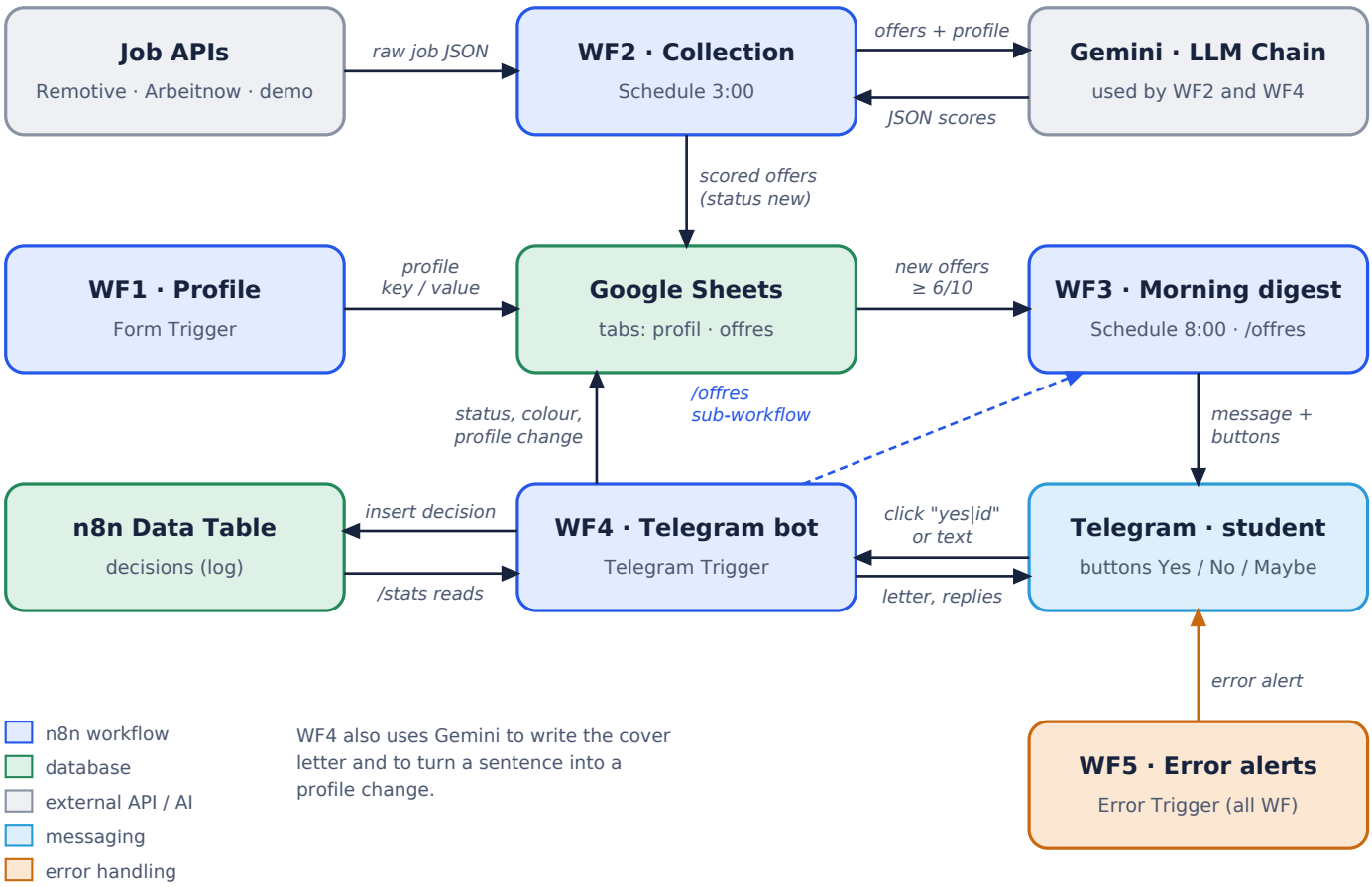
- **Schedule Trigger at 3:00**: collects new offers from job APIs and scores them with AI.
- **Schedule Trigger at 8:00**: sends the best new offers on Telegram (score ≥ 6/10, sorted by deadline, max 5).
- **Telegram Trigger**: reacts to every button click (Yes / No / Maybe) and every message (commands or profile changes in plain language).
- **Form Trigger**: the student fills in their profile once (studies, GPA, languages, countries, cities, fields, contract type).

On-demand testing

- "Execute workflow" on the collection workflow adds 4 demo offers (3 relevant, 1 irrelevant) with unique IDs, so the whole pipeline can be tested without waiting for real offers.
- Sending **/offres** to the Telegram bot runs the morning digest instantly.
- Clicking the buttons, or writing "Add Lisbon to my cities" to the bot, tests the Telegram workflow live.

3. Architecture diagram

Five coordinated workflows share a Google Sheets database (tabs *profil* and *offres*) and an n8n Data Table (*decisions*).



Workflow details

WF1 · Profile

Form Trigger → Read current profile → Clear old profile → Code: one row per answer, merge languages, keep Telegram chat_id → Save to Sheets

Travels: form answers → rows {key, value}

WF2 · Collection (3:00)

Schedule → Read profile → [Remotive API | Arbeitnow API | Demo offers] → Code: normalize each source → Merge → If: valid offer → Read known offers → Code: remove duplicates, keep matching fields, drop excluded words, max 15 → Code: batch → LLM Chain (Gemini): score 0–10 + summary + reason → Code: parse and check JSON → Save to Sheets (status “new”)

Travels: raw job JSON → normalized offer {id, title, company, location, contract, duration, salary, deadline, url} → scored offer {+ score, summary, reason, status}

WF3 · Morning digest (8:00 or /offres)

Schedule / Manual / called by WF4 → Read profile + offers → Code: status “new” and score ≥ 6, sort by deadline then score, max 5 → If: any offer? → Telegram message with buttons → Sheets: status “sent”

Travels: offer + chat_id → Telegram message with callback data “action|offer_id”

WF4 · Telegram bot

Telegram Trigger → Code: read update → Switch: message / offer click / profile confirmation. **Message** → Code: identify user and command → Switch (10 routes): /aide, /profil, /liens, /offres (calls WF3 as sub-workflow with inputs chat_id and origine), /avoir, /stats (reads the decisions table), /panne (live error test), unknown command → usage message, free text → LLM Chain turns the sentence into profile changes → confirmation buttons. **Offer click** → find offer by id → Google Sheets API: colour the row (green / orange / grey) → update status → Data Table: insert decision → Switch: Yes → LLM Chain writes a cover letter → Telegram; No → delete the Telegram message; Maybe → row stays orange. **Profile confirmation** → apply or cancel the pending change → confirm on Telegram

Travels: callback “yes|id” → offer row → letter text; free text → JSON {key, new value} → profile row

WF5 · Error alerts

Error Trigger (any workflow fails) → Read chat_id → Telegram alert with workflow name, failing node and error message

Travels: execution error → alert message

What travels between blocks

From → To	Data passed
Job APIs → WF2	Raw offers (JSON), different format for each source
WF2 → Gemini → WF2	Batch of normalized offers + profile; back: JSON array {id, score, summary, reason}
WF2 → Google Sheets	Scored offers with status “new” and date added
WF1 → Google Sheets	Profile rows {key, value}, Telegram chat_id preserved
Google Sheets → WF3	New offers scored ≥ 6/10, chat_id
WF3 → Telegram	One message per offer, buttons with callback “action|offer_id”
Telegram → WF4	Button clicks (action + offer id) and text messages
WF4 → Google Sheets	New status, decision date, row colour, confirmed profile changes
WF4 ↔ Data Table decisions	Insert {offre_id, titre, entreprise, decision, score}; /stats reads it back
Any workflow → WF5 → Telegram	Workflow name, failing node, error message

4. Target level and cut line

Target: Level 2, with selected Level 3 features. The core project is complete when it can collect offers from an API, score them with AI against the profile, store them without duplicates, send the best ones on Telegram with buttons, update their status from the buttons, and alert on errors.

Core features (must keep)

- Profile form and persistent storage in Google Sheets.
- Scheduled collection from job APIs, normalization and deduplication.
- AI scoring with a strict JSON format and validation.
- Morning Telegram digest with Yes / No / Maybe buttons and status tracking.
- Retries, fallback handling and an error workflow.

Cut line — remove these first if time runs short

1. Gmail draft with the CV attached on “Yes” (built, currently switched off: the cover letter is sent on Telegram instead).
2. Profile editing in plain language and the extra commands (/profil, /liens, /avoir).
3. Row colouring in the sheet (the status column is enough).

Always kept: collection → AI scoring → database → Telegram digest with Yes / No / Maybe → status tracking → error handling.

5. Minimum technical requirements

Requirement	Where it is used
If or Set node	If “Valid offer?” (WF2), If “Any offer?” (WF3), If “Change understood?” (WF4), Set “My application” (WF4)

Requirement	Where it is used
Code node	Normalizing each source, removing duplicates, sorting by deadline, parsing the AI's JSON, reading Telegram updates
Router / If / Switch	Switch "Message or click?" (3 routes) and Switch "Which command?" (10 routes) in WF4
API or MCP connection	Remotive and Arbeitnow job APIs (HTTP Request), Google Sheets API (row colouring), Telegram Bot API, Gemini API
Message trigger	Telegram Trigger (button clicks and messages)
Database	n8n Data Table "decisions" (written on every click, read back by /stats in another run); Google Sheets "profil" and "offres" tabs as the shared business table
AI node	Basic LLM Chain with Gemini and a written system prompt: scoring offers (WF2), writing cover letters and understanding profile changes (WF4)
Sub-workflow	WF4 calls WF3 (Execute Workflow) with defined inputs chat_id and origine when the student sends /offres
Error handling & retries	Retry on fail on every API, Sheets and Telegram node; a failing source does not stop the others; an unreadable AI answer gets the status "AI error" instead of crashing; WF5 error workflow sends a Telegram alert with workflow name, node, error and execution URL; /panne triggers a failure live

Reliability rules and test plan

- Deduplicate by offer id and by title + company, so the same offer is never stored or sent twice.
- Track each offer with a status (new → sent → maybe / rejected / application), so no offer is notified twice.
- Validate the AI answer: strict JSON, matched to each offer by id; invalid answers are flagged, not lost.
- Test with a relevant offer, an irrelevant offer, a duplicate run, and a simulated API failure.
- Generate drafts and suggestions only: nothing is sent to an employer without the student's approval.

Expected result

A working n8n assistant that collects offers, scores them against the student's profile, delivers the best ones on Telegram every morning, lets the student decide in one tap, and prepares a tailored cover letter while keeping manual control over applying.